

what is data ingestion in pymongo? can you use pymongo to ingest whole excel sheets into a mongodb database?

Yes. In the MongoDB/PyMongo context, **data ingestion** simply means the process of importing data from an external source into MongoDB.

For example:

```
Excel File (.xlsx)
↓
Read with pandas/openpyxl
↓
Transform / clean data
↓
PyMongo inserts documents
↓
MongoDB Collection
```

Can PyMongo ingest entire Excel sheets?

Yes, but **PyMongo** itself does not read Excel files.

Typically you would:

1. Read the Excel file using `pandas` or `openpyxl`
2. Convert rows into Python dictionaries

3. Use PyMongo to insert those dictionaries into MongoDB

Example:

Python

Run

```
import pandas as pd
from pymongo import MongoClient

# Connect to MongoDB
client = MongoClient("mongodb://localhost:27017/")
db = client["my_database"]
collection = db["employees"]

# Read Excel sheet
df = pd.read_excel("employees.xlsx")

# Convert DataFrame to list of dictionaries
records = df.to_dict("records")

# Insert into MongoDB
collection.insert_many(records)

print(f"Inserted {len(records)} documents")
```

If your Excel sheet looks like:

id	name	department
1	Alice	HR
2	Bob	IT

MongoDB will store:

JSON

```
{
  "_id": ObjectId(...),
  "id": 1,
  "name": "Alice",
  "department": "HR"
}
```

JSON

```
{
  "_id": ObjectId(...),
  "id": 2,
  "name": "Bob",
  "department": "IT"
}
```

Multiple sheets

You can also ingest every sheet in a workbook:

Python

Run

```
excel = pd.ExcelFile("company_data.xlsx")

for sheet_name in excel.sheet_names:
    df = pd.read_excel(excel, sheet_name=sheet_name)

    records = df.to_dict("records")

    db[sheet_name].insert_many(records)
```

This would create one MongoDB collection per Excel sheet.

Things to watch out for

- **Data types:** Excel dates, NaN values, and formulas may need conversion.
- **Large files:** Use batching (`insert_many` in chunks) for very large spreadsheets.
- **Schema validation:** If your MongoDB collection uses JSON Schema validation, some rows may fail insertion.
- **Nested data:** Excel is tabular, while MongoDB is document-oriented. Sometimes you'll want to transform rows into nested documents before insertion.

For most projects, the common ingestion stack is:

Python

Run

Excel → pandas → data cleaning/transformation → PyMongo → MongoDB

PyMongo is responsible for the **MongoDB insertion layer**, while libraries like pandas handle the **Excel ingestion layer**.



The distinction is:

- **DNS issue** → Python cannot obtain the SRV records from DNS.
- **PyMongo issue** → DNS works, but PyMongo fails afterward.

Step 1: Test DNS independently

Run:

Python

Run

```
import dns.resolver

answers = dns.resolver.resolve(
    "_mongodb._tcp.cluster0.8pmfqqn.mongodb.net",
    "SRV"
)

for answer in answers:
    print(answer)
```

If this fails

For example:

```
dns.resolver.NoAnswer
```

or

```
dns.resolver.NXDOMAIN
```

then you have a DNS problem.

Possible fixes:

Upgrade dnspython

Bash

```
pip install -U dnspython
```

Change DNS servers

On Windows, try:

- Cloudflare: 1.1.1.1 , 1.0.0.1
- Google: 8.8.8.8 , 8.8.4.4

Then:

Bash

```
ipconfig /flushdns
```

Check VPN / corporate network

Some VPNs, university networks, and corporate DNS servers block SRV lookups.

Try:

- Different Wi-Fi
- Mobile hotspot
- VPN off

Use Atlas's non-SRV connection string

Atlas provides a standard URI that bypasses SRV lookups:

```
mongodb://host1:27017,host2:27017,host3:27017/?replicaSet=...
```

This often works even when SRV DNS resolution is broken.

Step 2: If DNS works

If:

Python

Run

```
dns.resolver.resolve(...)
```

returns records successfully, then DNS is not the problem.

Now test:

Python

Run

```
from pymongo import MongoClient

client = MongoClient(MONGO_URI)
print(client.admin.command("ping"))
```

If this fails

Common causes:

Old PyMongo

Python

Run

```
import pymongo
print(pymongo.version)
```

Upgrade:

Bash

```
pip install -U pymongo
```

Atlas IP whitelist

Atlas → Network Access

Add your current IP or temporarily:

```
0.0.0.0/0
```

for testing.

Bad credentials

You'll usually get:

```
Authentication failed
```

rather than a DNS error.

Wrong URI

Common mistakes:

```
mongodb+srv://user:pass@cluster0.mongodb.net
```

when the actual cluster is:

```
mongodb+srv://user:pass@cluster0-abcd.mongodb.net
```

Copy the URI directly from Atlas rather than typing it manually.

What your specific error suggests

Your original exception was:

```
The DNS response does not contain an answer to the question:
_mongodb._tcp.cluster0.8pmfqqn.mongodb.net
```

That error is thrown by the DNS resolution layer that PyMongo uses.

If `nslookup` works but PyMongo fails, I would specifically check:

1. `dnspython` version
2. Whether the exact SRV lookup works from Python
3. Whether the Python interpreter you're running is the same one where PyMongo and `dnspython` are installed

Run:

Python

Run

```
import sys
import pymongo
import dns

print(sys.executable)
print("PyMongo:", pymongo.version)
print("dnspython:", dns.__version__)
```

and

Python

Run

```
import dns.resolver

print(
    dns.resolver.resolve(
        "_mongodb._tcp.cluster0.8pmfqqn.mongodb.net",
        "SRV"
    )
)
```

Those outputs will usually reveal immediately whether you're dealing with a DNS configuration problem or a PyMongo environment/package problem.



ive determined it to be a dns problem as i am getting noanswer. even on cloudflare, it does not work. i have also tried to flush dns and changed to my hotspot. dnspython is at 2.8.0.

interpreter running is same one where pymongo and dnspython are installed, yes. no, srv lookup does not work from inside python (noanswer)

Given everything you've tested, I suspect the issue is not your local DNS configuration but the **hostname** itself.

The critical clue is:

```
nslookup works, but
dns.resolver.resolve("_mongodb._tcp.cluster0.8pmfqqn.mongodb.net", "SRV")
returns NoAnswer .
```

Those are not querying the same record.

When you run:

```
Bash
nslookup cluster0.8pmfqqn.mongodb.net
```

you're usually querying an **A record** (IP address).

PyMongo is querying:

```
_mongodb._tcp.cluster0.8pmfqqn.mongodb.net
```

which is an **SRV record**.

A hostname can have a valid A record but no SRV record.

Verify exactly what is failing

Try:

```
Bash
nslookup -type=SRV _mongodb._tcp.cluster0.8pmfqqn.mongodb.net
```

or:

Bash

```
nslookup  
set type=SRV  
_mongodb._tcp.cluster0.8pmfqqn.mongodb.net
```

If that returns:

```
*** No SRV records found
```

then the problem is not Python, PyMongo, or your DNS server.

It means that Atlas does not have an SRV record published for that hostname.

Most likely causes

1. Wrong hostname in your URI

This is the most common.

For example, suppose Atlas gave:

```
mongodb+srv://user:pass@legalcluster.abc123.mongodb.net/
```

but your code has:

```
mongodb+srv://user:pass@cluster0.8pmfqqn.mongodb.net/
```

The hostname resolves, but Atlas hasn't published SRV records there.

Copy the URI directly from Atlas:

1. Atlas Dashboard
2. Database
3. Connect
4. Drivers
5. Python

Use that exact URI.

2. Atlas cluster was deleted or recreated

Atlas hostnames are tied to specific clusters.

If the cluster was recreated, the hostname may have changed.

Again, compare your URI against the current Atlas "Connect" page.

3. You're using an old saved URI

I've seen this happen when:

- cluster is renamed
- project is migrated
- free cluster is recreated

The URI stored in `.env` no longer matches the current Atlas cluster.

Quick test

Paste only the hostname portion (not username/password), for example:

```
cluster0.8pmfqqn.mongodb.net
```

or the redacted URI:

```
mongodb+srv://***:***@cluster0.8pmfqqn.mongodb.net/?retryWrites=true&w=majority
```

Then I can tell whether it looks like a valid Atlas SRV hostname pattern.

Temporary workaround

If Atlas is giving you trouble with SRV records, go to:

Connect → Drivers → "Show more connection options"

and obtain the **standard connection string**:

```
mongodb://host1:27017,host2:27017,host3:27017/?replicaSet=...
```

This bypasses SRV lookups entirely and is a useful diagnostic. If the standard URI works immediately, you've confirmed the issue is specifically with SRV record resolution for the hostname you're using.



it works now with using the standard connection string.

thanks

Good, that narrows the problem down considerably.

Since the **standard connection string works**, you've confirmed that:

-  MongoDB Atlas is reachable
-  Your credentials are valid
-  Network access/IP whitelist is configured correctly
-  PyMongo itself is functioning
-  SRV record discovery (`mongodb+srv://`) is failing

At this point, there is no need to troubleshoot PyMongo further. The issue is specifically with resolving the Atlas SRV record.

For a development or personal project, it's perfectly acceptable to continue using the standard URI:

Python

Run

```
MONGO_URI = "mongodb://host1,host2,host3/?replicaSet=..."
```

PyMongo will still discover the replica set members and handle failover correctly because the replica set information is embedded in the connection string.

One thing I'd still recommend checking later is the Atlas-generated SRV URI from the Atlas dashboard and comparing its hostname to the one you were using. If they differ, you've identified the root cause. If they are identical, then there may be an issue with the SRV records published for that specific cluster or project.

For your data-ingestion workflow (Excel → pandas → PyMongo → MongoDB), using the standard connection string won't affect anything else in your code. Your existing `insert_many()` logic can remain exactly as-is.

